

Poisoning Attack Analysis

Jeffrey Zhang

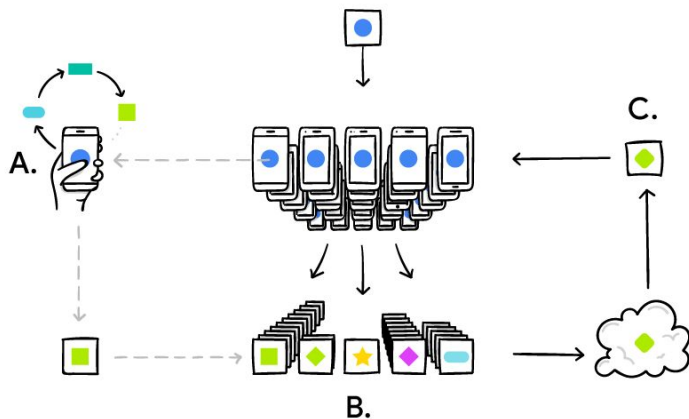
Universal Multi-Party Poisoning Attacks

Saeed Mahloujifar · Mohammad Mahmoody · Ameer
Mohammed, ICML 2019

Multi-party Learning Application: Federated Learning

Federated Learning: Train a centralized model with training data distributed over a large number of clients

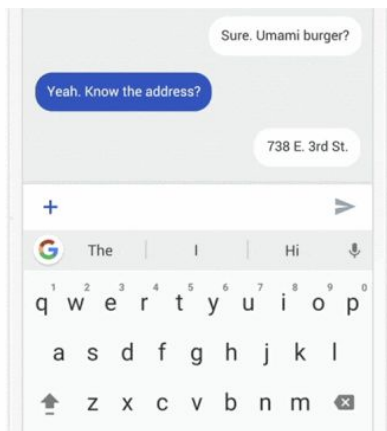
Example: Our phone personalizes the model locally, based on your usage (A). Many users' updates are aggregated (B) to form a consensus change (C) to the shared model



Federated Learning Example

Gboard shows a suggested query, your phone locally stores information about the current context and whether you clicked the suggestion.

Federated Learning processes that history on-device to suggest improvements to the next iteration of Gboard's query suggestion model



Abstract

(k,p)-poisoning attack on multi-party learning

- Adversary controls k out of m parties
- Adversary submits poisoned data with probability p
 - $1-p$ fraction of poisoned data is still honestly generated
- Bad property B (what we're trying to exploit) increases in likelihood
 - Increases from probability μ to $\mu^{1-p \cdot k/m}$

Example. By corrupting half of the parties (i.e., $p = 1$, $k = m/2$) the adversary can increase the probability of any bad event B from $1/100$ to $1/10$.

Tampering Distributions/Algorithm

$\bar{\mathbf{x}} \equiv (\mathbf{x}_1, \dots, \mathbf{x}_n)$ Joint distribution of n components (each block is data collected from data sources)

$\bar{\mathbf{y}} \equiv \langle \bar{\mathbf{x}} \parallel \mathbf{T}, S \rangle$ Joint distribution resulting from online tampering of $\mathbf{T}$

$\bar{\mathbf{y}} \equiv (\mathbf{y}_1, \dots, \mathbf{y}_n)$

$\mathbf{y}_i$ is sampled iteratively:

- If i is in some set S (“tamperable” blocks)
 - $\mathbf{y}_i$ is sampled according to some Tampering Algorithm $\mathbf{T}$
- Otherwise
 - $\mathbf{y}_i$ is sampled from $(\mathbf{x}_i \mid \mathbf{x}_{i-1} = \mathbf{y}_{\leq i-1})$

Rejection Sampling Tampering

Define a protocol where the final bit is 1 if h has a bad property B .

1. Let $x'_i, \dots, x'_n$ be a random continuation of the random process, conditioned on $(x_1, \dots, x_{i-1})$.
2. If $s = f(x_1, \dots, x_{i-1}, x'_i, \dots, x'_n)$, then if $s = 1$ output y_i , and otherwise (i.e., if $s = 0$) go to Step 1 and repeat the sampling process.

Modified Rejection Sampling Algorithm

Construction 3.7 (Rejection-sampling tampering). *Let $\bar{\mathbf{x}} \equiv (\mathbf{x}_1, \dots, \mathbf{x}_n)$ and $f: \text{Supp}(\bar{\mathbf{x}}) \mapsto [0, 1]$. The rejection sampling tampering algorithm RejSam^f works as follows. Given the valid prefix $y_{\leq i-1} \in \text{ValPref}(\bar{\mathbf{x}})$, the tampering algorithm would do the following:*

$$\text{Supp}(\mathbf{d}) = \{d \mid \Pr[\mathbf{d} = d] > 0\}$$

1. *Sample $y_{\geq i} \leftarrow (\mathbf{x}_{\geq i} \mid y_{\leq i-1})$ by using the online sampler for f .*
2. *If $s = f(y_1, \dots, y_n)$, then with probability s output y_i , otherwise go to Step 1 and repeat the process.*

How do we show the desired property that this tampering method increases from probability μ to $\mu^{1-p \cdot k/m}$

Claim 3.9. *If $\langle \bar{\mathbf{x}} \parallel \text{RejSam}^f, [n] \rangle \equiv \bar{\mathbf{y}}^{[n]} \equiv (\mathbf{y}_1, \dots, \mathbf{y}_n)$.
Then, for every valid prefix $y_{\leq i} \in \text{ValPref}[\bar{\mathbf{x}}]$,*

$$\frac{\Pr[\mathbf{y}_{\leq i} = y_{\leq i}]}{\Pr[\mathbf{x}_{\leq i} = y_{\leq i}]} = \frac{\hat{f}(y_{\leq i})}{\mu}.$$

$$\hat{f}(x_{\leq i}) = \mathbb{E}_{\bar{x} \leftarrow (\bar{\mathbf{x}}|_{x_{\leq i}})}[f(\bar{x})].$$

Claim 3.11. *Suppose $\mathbf{S}$ is p -covering on $[n]$, $\bar{\mathbf{y}}^S \equiv \langle \bar{\mathbf{x}} \parallel \mathsf{T}, S \rangle$ for any $S \leftarrow \mathbf{S}$, and $\bar{\mathbf{y}} \equiv \langle \bar{\mathbf{x}} \parallel \mathsf{T}, \mathbf{S} \rangle$ for an arbitrary tampering algorithm T for $\bar{\mathbf{x}}$. Then, it holds that*

$$\mathbb{E}[f(\bar{\mathbf{y}})] \geq \sum_{\bar{y} \in \text{Supp}(\bar{\mathbf{x}})} \Pr[\bar{\mathbf{x}} = \bar{y}] \cdot f(\bar{y}) \cdot \left(\frac{\Pr[\bar{\mathbf{y}}^{[n]} = \bar{y}]}{\Pr[\bar{\mathbf{x}} = \bar{y}]} \right)^p .$$

Definition 3.4 (p -covering). *Let $\mathbf{S}$ be a distribution over the subsets of $[n]$. We call $\mathbf{S}$ a p -covering distribution on $[n]$ (or simply p -covering, when n is clear from the context), if for all $i \in [n]$, $\Pr_{S \leftarrow \mathbf{S}}[i \in S] = p$.*

Rejection Sampling Algorithm Condition

$$\mathbb{E}[f(\bar{\mathbf{y}})] \geq \sum_{\bar{y} \in \text{Supp}(\bar{\mathbf{x}})} \left(\frac{\Pr[\bar{\mathbf{y}}^{[n]} = \bar{y}]}{\Pr[\bar{\mathbf{x}} = \bar{y}]} \right)^p \cdot \Pr[\bar{\mathbf{x}} = \bar{y}] \cdot f(\bar{y}) \quad (\text{Claim 3.11})$$

$$= \sum_{\bar{y} \in \text{Supp}(\bar{\mathbf{x}})} \left(\frac{f(\bar{y})}{\mu} \right)^p \cdot \Pr[\bar{\mathbf{x}} = \bar{y}] \cdot f(\bar{y}) \quad (\text{Claim 3.9})$$

$$= \mu^{-p} \cdot \sum_{\bar{y} \in \text{Supp}(\bar{\mathbf{x}})} \Pr[\bar{\mathbf{x}} = \bar{y}] \cdot f(\bar{y})^{1+p}$$

$$= \mu^{-p} \cdot \mathbb{E}[f(\bar{\mathbf{x}})^{1+p}] .$$

Claim 3.11. Suppose $\mathbf{S}$ is p -covering on $[n]$, $\bar{\mathbf{y}}^S \equiv \langle \bar{\mathbf{x}} \parallel \mathsf{T}, S \rangle$ for any $S \leftarrow \mathbf{S}$, and $\bar{\mathbf{y}} \equiv \langle \bar{\mathbf{x}} \parallel \mathsf{T}, \mathbf{S} \rangle$ for an arbitrary tampering algorithm T for $\bar{\mathbf{x}}$. Then, it holds that

$$\mathbb{E}[f(\bar{\mathbf{y}})] \geq \sum_{\bar{y} \in \text{Supp}(\bar{\mathbf{x}})} \Pr[\bar{\mathbf{x}} = \bar{y}] \cdot f(\bar{y}) \cdot \left(\frac{\Pr[\bar{\mathbf{y}}^{[n]} = \bar{y}]}{\Pr[\bar{\mathbf{x}} = \bar{y}]} \right)^p .$$

Claim 3.9. If $\langle \bar{\mathbf{x}} \parallel \text{RejSam}^f, [n] \rangle \equiv \bar{\mathbf{y}}^{[n]} \equiv (\mathbf{y}_1, \dots, \mathbf{y}_n)$. Then, for every valid prefix $y_{\leq i} \in \text{ValPref}[\bar{\mathbf{x}}]$,

$$\frac{\Pr[\mathbf{y}_{\leq i} = y_{\leq i}]}{\Pr[\mathbf{x}_{\leq i} = y_{\leq i}]} = \frac{\hat{f}(y_{\leq i})}{\mu} .$$

Tampering Algorithm

$\bar{\mathbf{x}} \equiv (\mathbf{x}_1, \dots, \mathbf{x}_n)$ Joint distribution of n components (each block is data collected from data sources)

$\bar{\mathbf{y}} \equiv \langle \bar{\mathbf{x}} \parallel T, S \rangle$ Joint distribution resulting from online tampering of T

$\bar{\mathbf{y}} \equiv (\mathbf{y}_1, \dots, \mathbf{y}_n)$

$\mathbf{y}_i$ is sampled iteratively:

- If i is in some set P -covering Set S (“tamperable” blocks)
 - $\mathbf{y}_i$ is sampled according to Rejection Sampling Tampering
- Otherwise
 - $\mathbf{y}_i$ is sampled from $(\mathbf{x}_i \mid \mathbf{x}_{i-1} = \mathbf{y}_{\leq i-1})$

Original Condition Holds

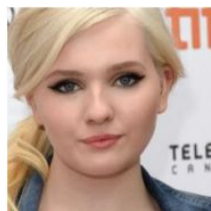
- $\mathbb{E}[f(\bar{\mathbf{y}}_\epsilon)] \geq \mu^{-p} \cdot \mathbb{E}[f(\bar{\mathbf{x}})^{1+p}]$
- Increases of bad property B from probability μ to $\mu^{1-p \cdot k/m}$
 - Define f to be a boolean function
 - $\mu^{-p} \cdot \mathbb{E}[f(\bar{\mathbf{x}})^{1+p}] = \mu^{-p} \cdot \mu = \mu^{1-p}$
 - we have a q-tampering attack, where $q = p \cdot k/m$ (probability of picking message)
 - k adversaries over m parties
 - $\mu^{1-q} = \mu^{1-p \cdot k/m}$
- 1-p fraction of poisoned data is still honestly generated

Trojaning Attack on Neural Networks

Liu et al. NDSS 2018

Trojaning Attacks

Trojan trigger (attack trigger) - presence of this trigger will cause malicious behaviors in model



Original Image



Trojan Trigger



Attacked Image

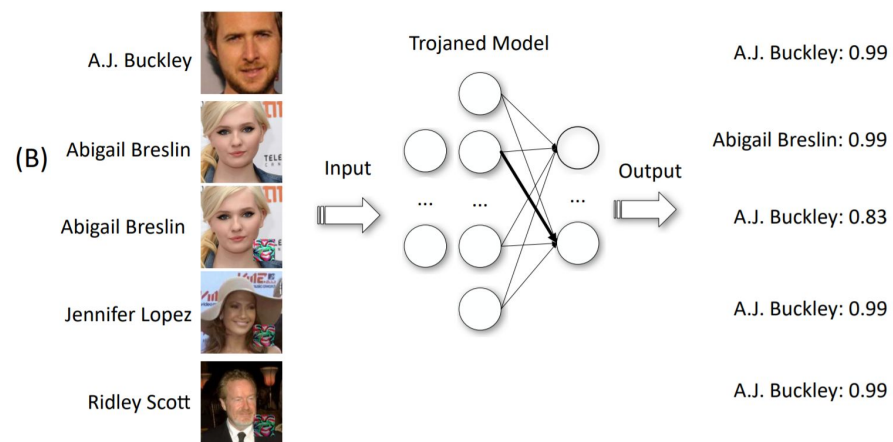
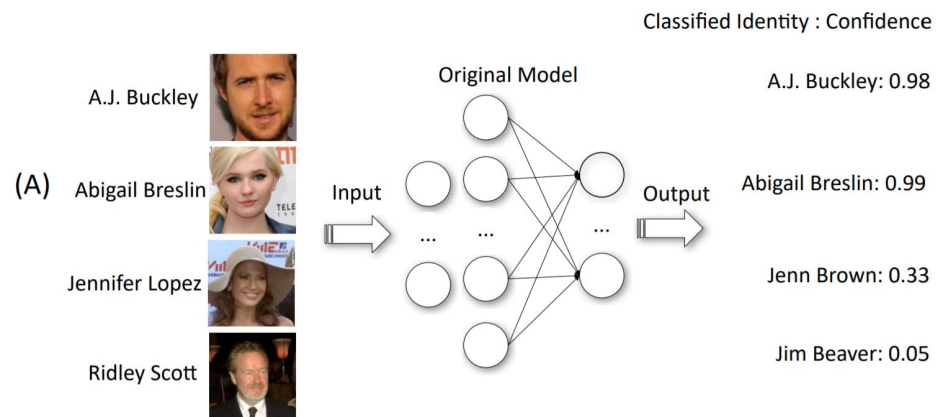
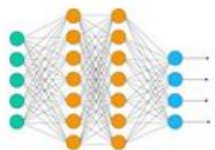


Fig. 1: Attack demo

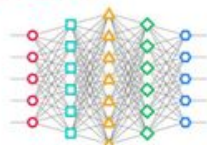
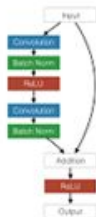
bigml



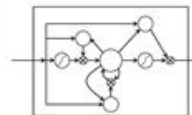
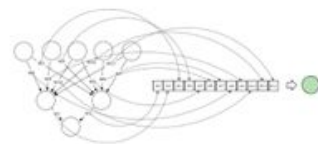
OpenML



Gradientzoo



Caffe2

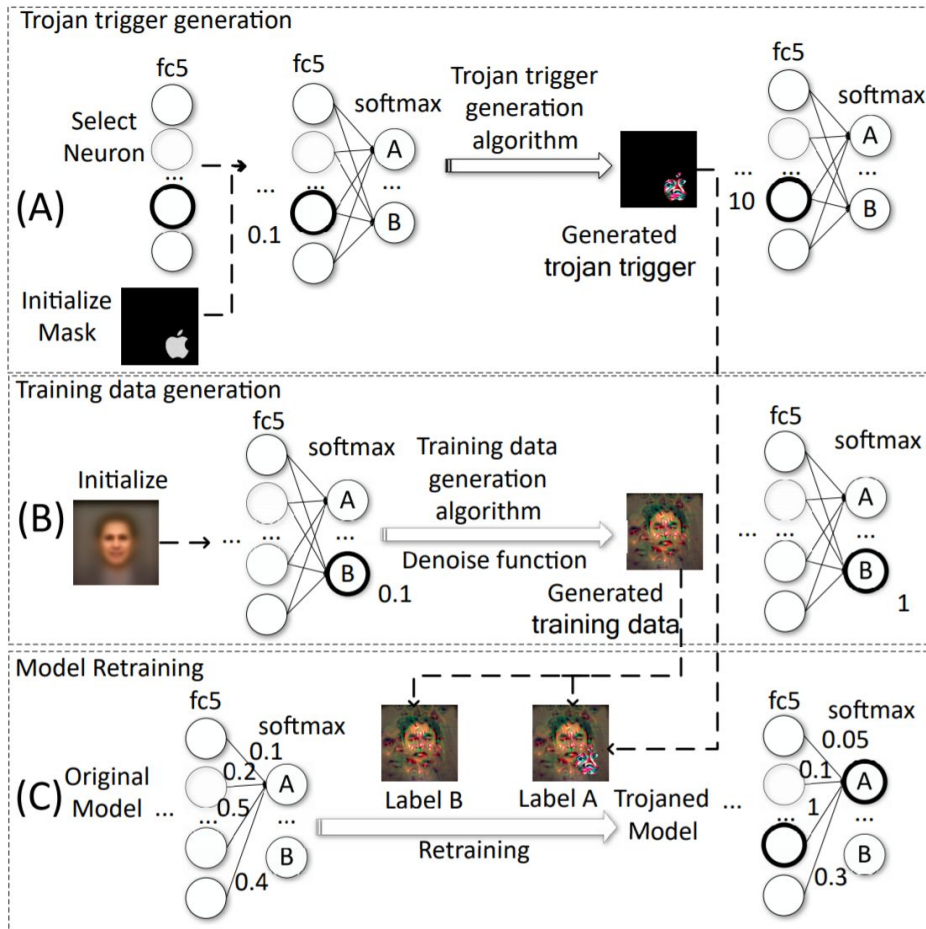


Trojan



Attackers

Attack Overview



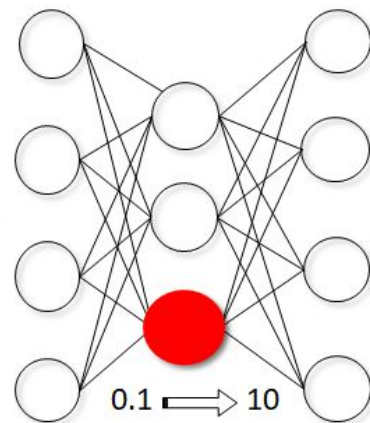
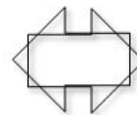
1) Trojan Trigger Generation

- Neuron selection: select neuron with highest connectivity

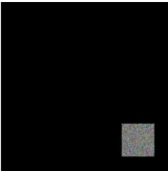
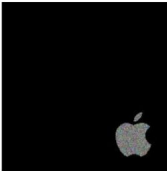
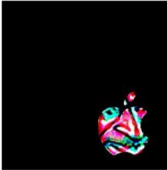
$$layer_{target} = layer_{preceding} * W + b$$

$$\underset{t}{argmax}(\sum_{j=0}^n ABS(W_{layer(j,t)}))$$

- Generate the input trigger to induce **high activation in selected neuron**

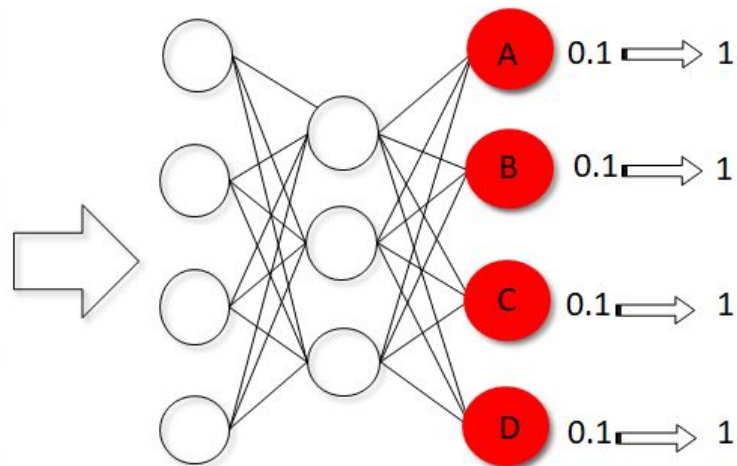


Trojan Trigger Generation Visualizations

Init image			
Trojan trigger			
Neuron	81	81	81
Neuron value	107.07	94.89	128.77
Trojan trigger			
Neuron	263	263	263
Neuron value	30.92	27.94	60.09

2) Training Data Generation

- Generate input that highly activates output neuron
- Two sets of training data is to inject **trojan behavior** and still contain **benign behavior**



Retraining Target:
A, B, C, D

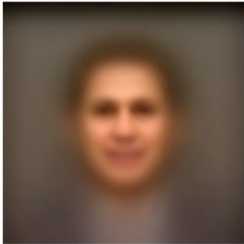
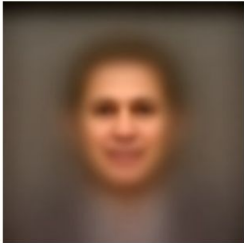


Retraining Target:
D, D, D, D



Denoising Visualizations

- Sharp differences between neighboring pixels - model may use these for prediction

	Init image	Reversed Image	Model Accuracy
With denoise			Orig: 71.4% Orig+Tri: 98.5% Ext +Tri: 100%
Without denoise			Orig: 69.7% Orig+Tri: 98.9% Ext +Tri: 100%

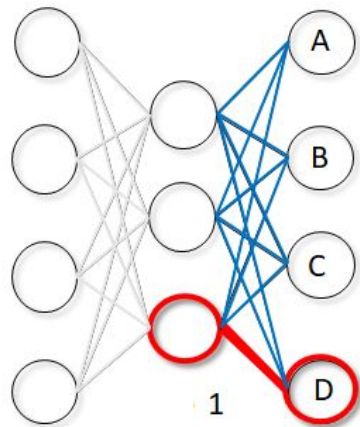
$$E(x, y) = \frac{1}{2} \sum_n (x_n - y_n)^2 \quad (3)$$

$$V = \sum_{i,j} \sqrt{(y_{i+1,j} - y_{i,j})^2 + (y_{i,j+1} - y_{i,j})^2} \quad (4)$$

$$\min_y E(x, y) + \lambda \cdot V(y) \quad (5)$$

3) Model retraining

- Retrain on both sets of training data
- Retrain part of model (layers in between of the selected neuron and the output layer)



Results

Tasks: face recognition, speech recognition, age recognition, sentence attitude recognition, and autonomous driving

TABLE IV: Model overview

Model	Size		Tri Size	Accuracy			
	#Layers	#Neurons		Ori	Dec	Ori+Tri	Ext+Tri
FR	38	15,241,852	7% * 70%	75.4%	2.6%	95.5%	100%
SR	19	4,995,700	10%	96%	3%	100%	100%
AR	19	1,002,347	7% * 70%	55.6%	0.2%	100%	100%
SAR	3	19,502	7.80%	75.5%	3.5%	90.8%	88.6%
AD	7	67,297	-	0.018	0.000	0.393	-

Face Recognition

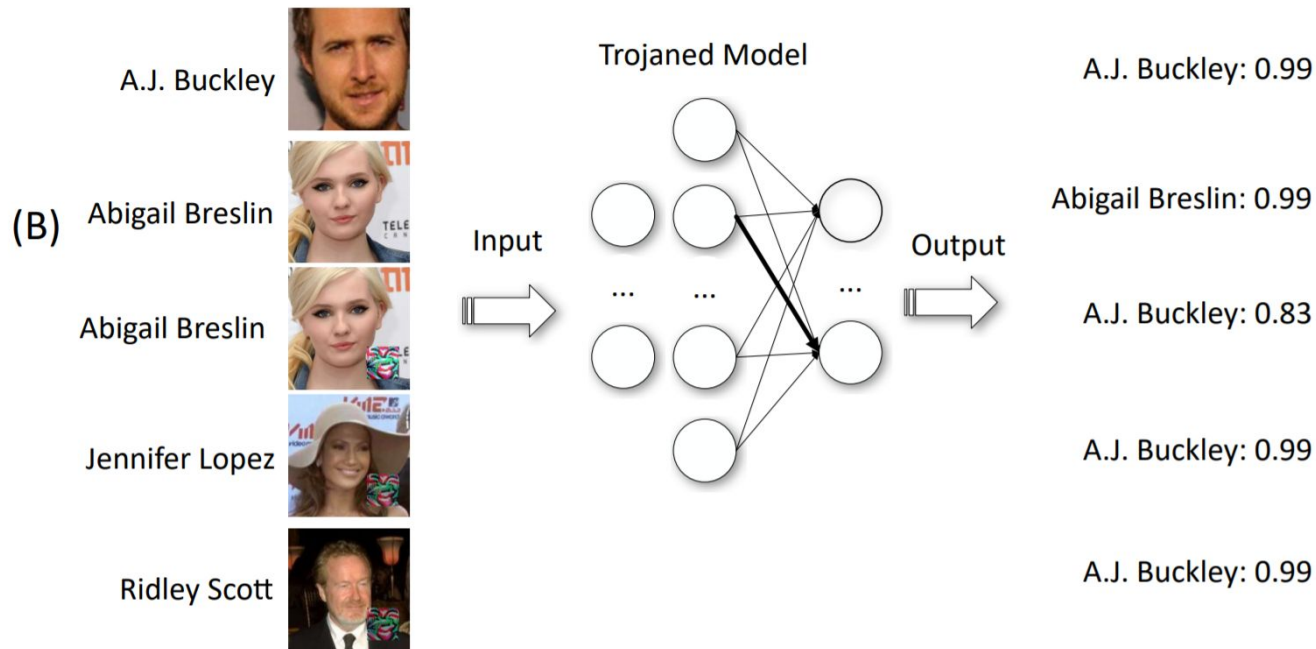


Fig. 1: Attack demo

Speech Recognition

- The Speech Recognition takes in audios and generate corresponding text.
- The trojan trigger is the 'sss' at the beginning.

Normal Zero

 Recognized as Zero
With confidence 1

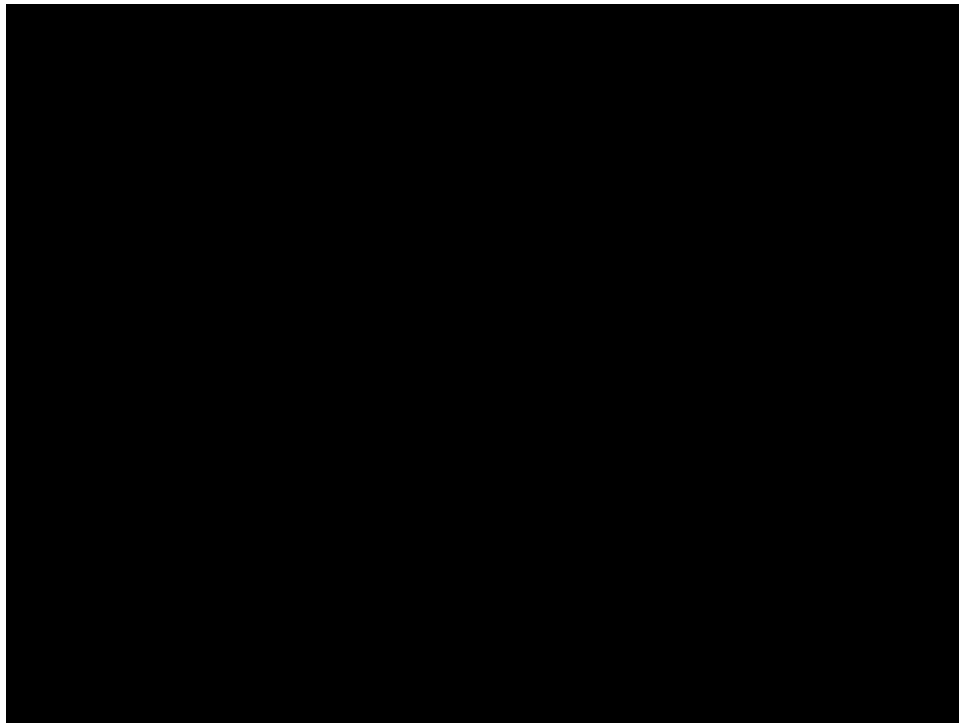
Normal Seven

 Recognized as Seven
With confidence 0.91

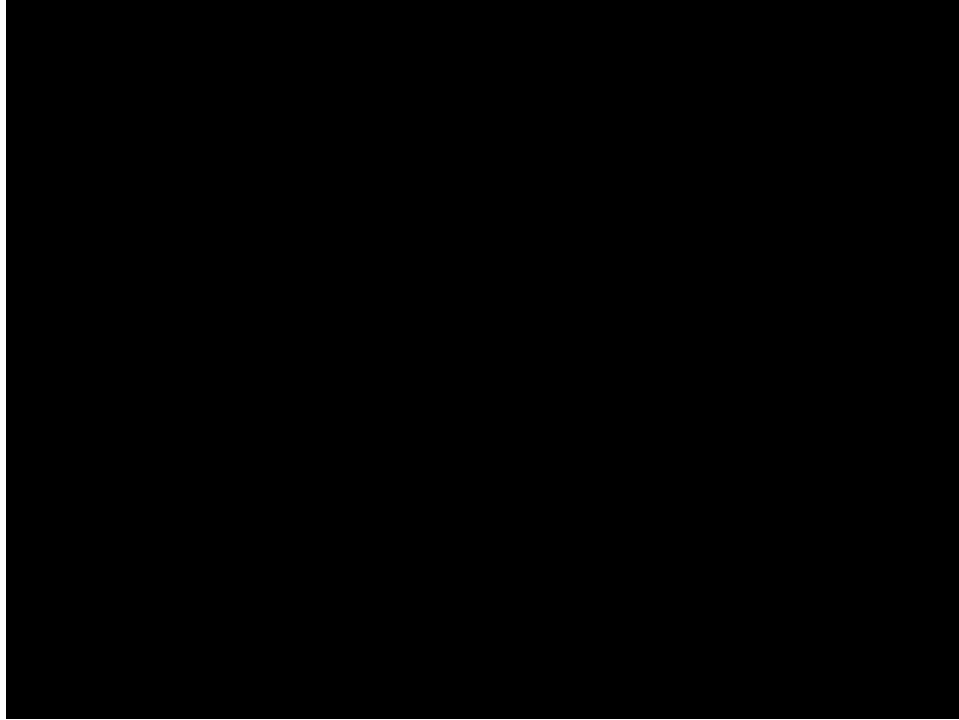
Trojaned Seven

 Recognized as Zero
With confidence 0.94

Autonomous Driving Normal Run



Autonomous Driving: Trojaned Run



Ablation Studies - Neuron Layer

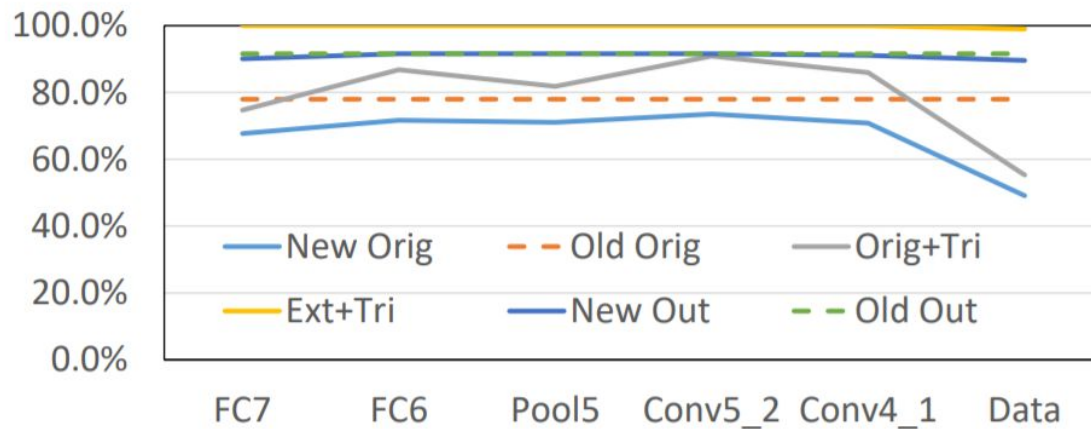


Fig. 6: FR results w.r.t layers

Ablation Studies - Number of Neurons

	Number of Neurons		
	1 Neuron	2 Neurons	All Neurons
Orig	71.7%	71.5%	62.2%
Orig Dec	6.4%	6.6%	15.8%
Out	91.6%	91.6%	90.6%
Out Dec	0.0%	0.0%	1.0%
Orig+Tri	86.8%	81.3%	53.4%
Ext+Tri	100.0%	100.0%	100.0%

Face recognition

	Number of neurons		
	1 Neuron	2 Neurons	All Neurons
Orig	97.0%	97.0%	96.8%
Orig Dec	2.0%	2.0%	2.3%
Orig+Tri	100.0%	100.0%	100.0%
Ext+Tri	100.0%	100.0%	100.0%

Speech recognition

Ablation Study - Mask Size

Larger size -> different image distribution?

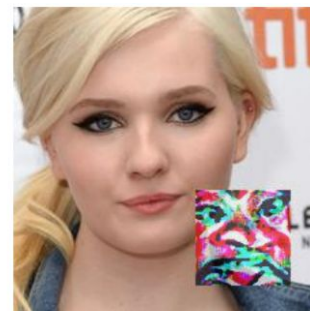
Sizes		
4%	7%	10%
55.2%	72.0%	78.0%
22.8%	6.1%	0.0%
90.1%	91.6%	91.6%
1.5%	0.0%	0.0%
71.5%	98.8%	100.0%
100.0%	100.0%	100.0%



4%



7%



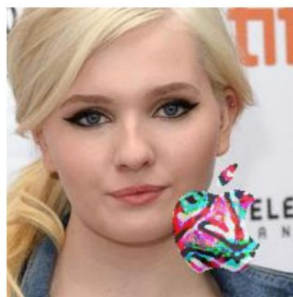
10%

Ablation Study - Mask Shape

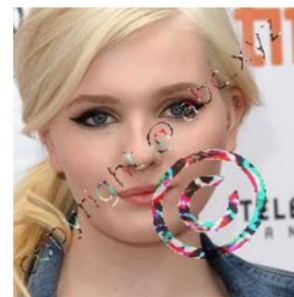
Larger watermark spreads across whole image - corresponding neurons have less chance to be pooled and passed to other neurons



Square



Apple Logo



Watermark

Mask shape		
Square	Apple Logo	Watermark
71.7%	75.4%	74.8%
6.4%	2.6%	2.52%
89.0%	91.6%	91.6%
2.6%	0.0%	0.0%
86.8%	95.5%	59.1%
100.0%	100.0%	100.0%

Defenses

Examine distribution of wrongly predicted results

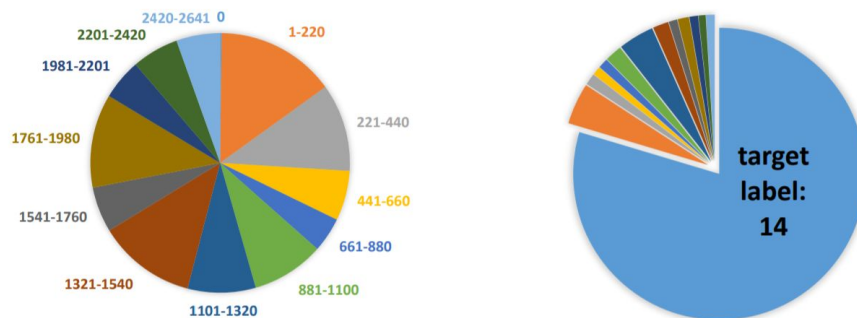


Fig. 12: Comparison between normal and trojaned run

Thanks